

Database Systems		
Lab #:	10	
Lab Title:	Implementation of different types of operators in SQL	
Submitted by:		
Names		Registration #
Sumaira Safeer		FA22-BCE-019

Rubrics name & number					Marks	
					In-Lab	Post-Lab
Engineering Knowledge	R2: Use of Engineering Knowledge and follow Experiment Procedures: Ability to follow experimental procedures, control variables, and record procedural steps on lab report.					
Problem Analysis	R6: Experimental Data Analysis: Ability to interpret findings, compare them to values in the literature, identify weaknesses and limitations.					
Design	R8: Best Coding Standards: Ability to follow the coding standards and programming practices.					
Modern Tools Usage	R9: Understand Tools: Ability to describe and explain the principles behind and applicability of engineering tools.					
Individual and Teamwork	R12: Individual Work Contributions: Ability to carry out individual responsibilities.					
	R13: Management of Team Work: Ability to appreciate, understand and work with multidisciplinary team members.					
Rubrics #	R2	R6	R8	R9	R12	R13
In Lab						
Post- Lab						

Lab No: 10

Title: Implementation of different types of operators in SQL.

- Arithmetic Operator.
- Logical Operator.
- Comparison Operator.
- Special Operator.
- Set Operator.

Objective:

To understand the working of SELECT queries and use different conditions involving logical and arithmetic operators to get the user desired results through queries.

In lab Tasks:

Design a table 'Customer'.

Table:



Table Name:

customer

Schema: customers

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	De
 Customer_ID	INT	☒	☒	☐	☐	☐	☐	☐	☐	
 Customer_Name	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
 Customer_City	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
 Grade	INT	☐	☒	☐	☐	☐	☐	☐	☐	
 Opening_Amount	INT	☐	☒	☐	☐	☐	☐	☐	☐	
 Recieving_Amount	INT	☐	☒	☐	☐	☐	☐	☐	☐	
 Payment_Amount	INT	☐	☒	☐	☐	☐	☐	☐	☐	
 Phone	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
		☐	☐	☐	☐	☐	☐	☐	☐	

Insertion of data:

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

Customer_ID	Customer_Name	Customer_City	Grade	Opening_Amount	Receiving_Amount	Payment_Amount	Phone
1	Sumaira Safeer	Islamabad	35	4300	1550	450	0333-754468
2	Aazan Zarsham	New York	2	4500	4300	2000	0308-753578
3	Hannah Zoe	Mianwali	23	76000	6500	900	0333-753957
4	Saqib Hassan	New York	1	5600	23500	54500	0344-863579
5	Tayyaba Khan	Multan	87	3400	4500	5400	0300-765367
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Task1:

Display customer id customer name and summation of opening amount and receiving amount where this summation is greater than 10,000.

oder_details
product
customer
product
oder_details
product
oder

Limit to 1000 rows

```

1 • SELECT * FROM customers.customer;
2 • SELECT
3     Customer_ID,
4     Customer_Name,
5     Opening_Amount + Receiving_Amount AS TotalAmount
6 FROM
7     customer
8 WHERE
9     Opening_Amount + Receiving_Amount > 10000;

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Customer_ID	Customer_Name	TotalAmount
3	Hannah Zoe	82500
4	Saqib Hassan	29100

29 14:45:55

SELECT Customer_ID, Customer_Name, Opening_Amount + Receiving_Amount

Error Code: 1146. Table 'university.customer' doesn't exist

0.000 sec

30 14:46:55

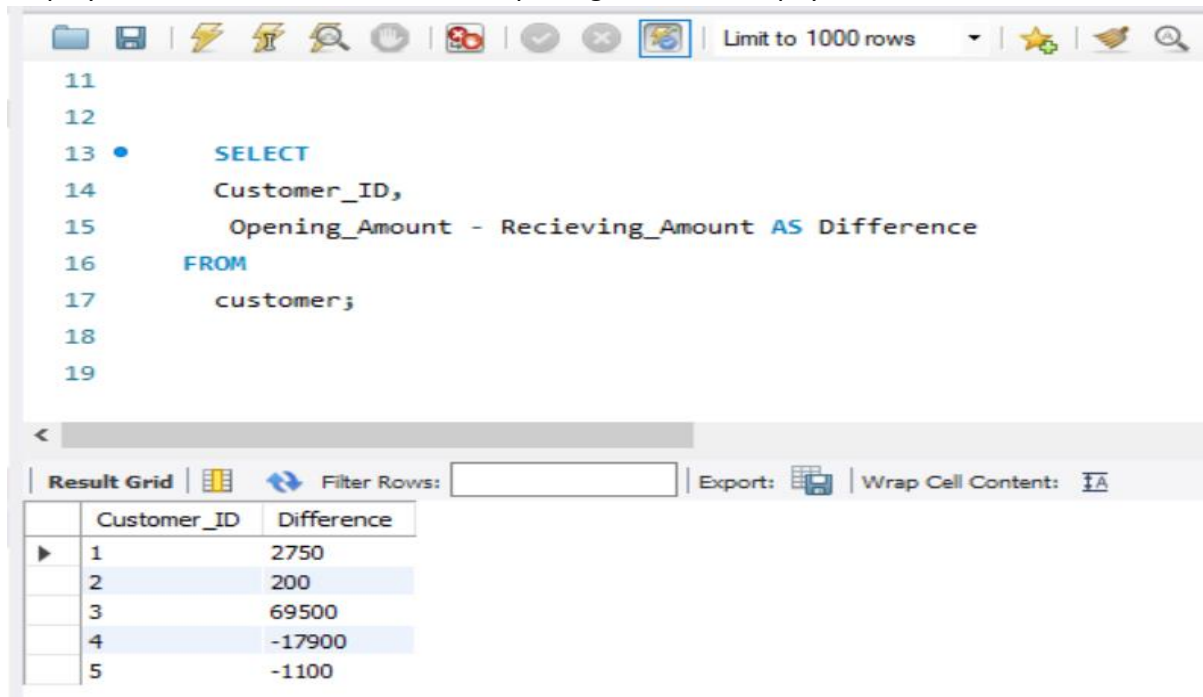
SELECT Customer_ID, Customer_Name, Opening_Amount + Receiving_Amount

2 row(s) returned

0.000 sec / 0.000 sec

Task 2:

Display customer id and difference of opening amount and payment amount.



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

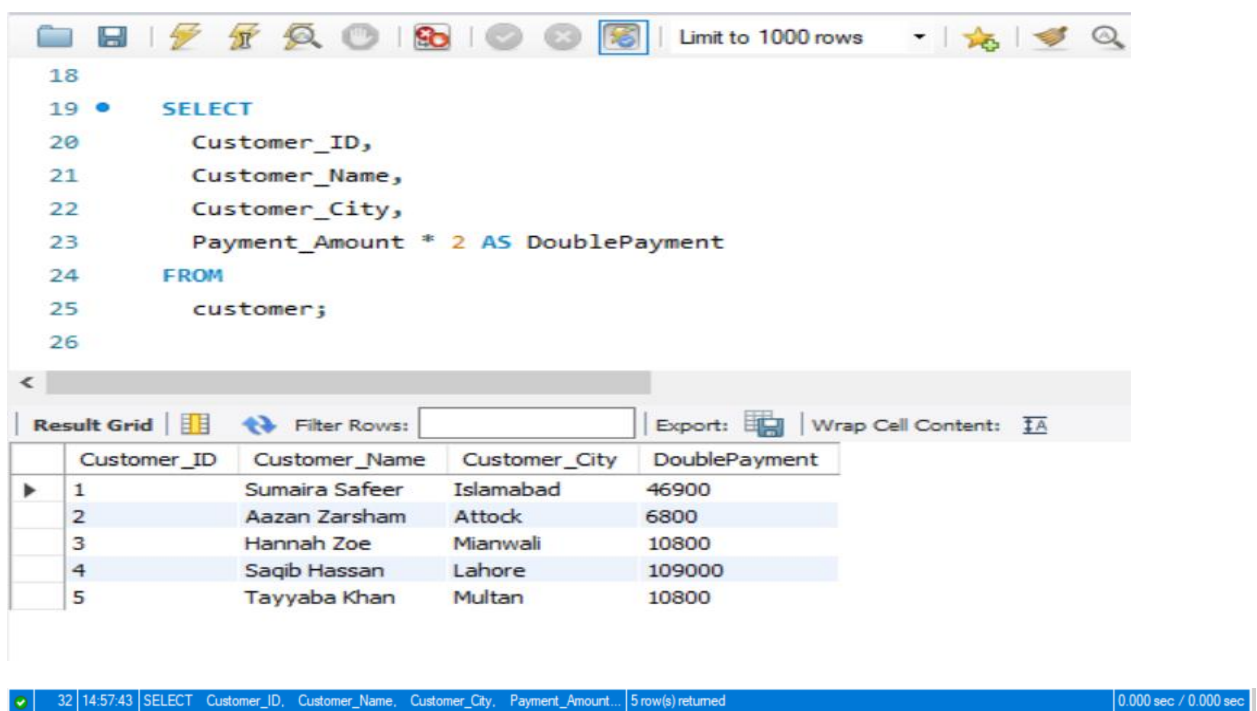
```
11
12
13 •   SELECT
14       Customer_ID,
15       Opening_Amount - Recieving_Amount AS Difference
16 FROM
17     customer;
18
19
```

Below the query editor, the 'Result Grid' is displayed with the following data:

	Customer_ID	Difference
▶	1	2750
	2	200
	3	69500
	4	-17900
	5	-1100

Task 3:

Display customer id, customer name, customer city and payment amount multiplied with 2.



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
18
19 •   SELECT
20       Customer_ID,
21       Customer_Name,
22       Customer_City,
23       Payment_Amount * 2 AS DoublePayment
24 FROM
25     customer;
26
```

Below the query editor, the 'Result Grid' is displayed with the following data:

	Customer_ID	Customer_Name	Customer_City	DoublePayment
▶	1	Sumaira Safeer	Islamabad	46900
	2	Aazan Zarsham	Attock	6800
	3	Hannah Zoe	Mianwali	10800
	4	Saqib Hassan	Lahore	109000
	5	Tayyaba Khan	Multan	10800

At the bottom of the screenshot, a status bar shows: 32 | 14:57:43 | SELECT Customer_ID, Customer_Name, Customer_City, Payment_Amount... | 5 row(s) returned | 0.000 sec / 0.000 sec

Task 4:

Display all records of customer where payment amount is equal to 2000.

The screenshot shows a database query editor with a toolbar at the top. The SQL query is as follows:

```
28 • SELECT
29     *
30 FROM
31     customer
32 WHERE
33     Payment_Amount = 2000;
34
```

Below the query, the 'Result Grid' is displayed with the following columns: Customer_ID, Customer_Name, Customer_City, Grade, Opening_Amount, Recieving_Amount, Payment_Amount, and Phone. The results show one record where Payment_Amount is 2000.

Customer_ID	Customer_Name	Customer_City	Grade	Opening_Amount	Recieving_Amount	Payment_Amount	Phone
2	Aazan Zarsham	Attock	45	4500	4300	2000	0308-753578

✓ 33 15:00:47 SELECT * FROM customer WHERE Payment_Amount = 2000 LIMIT 0, 1000 0 row(s) returned 0.000 sec / 0.000 sec

Task 5:

Display all records of customer where payment amount is greater than, less than, greater than and equal to, less than and equal to, not equal to 2000.

- **Greater than 2000:**

The screenshot shows a database query editor with a toolbar at the top. The SQL query is as follows:

```
36
37 • SELECT
38     *
39 FROM
40     customer
41 WHERE
42     Payment_Amount > 2000;
43
```

Below the query, the 'Result Grid' is displayed with the following columns: Customer_ID, Customer_Name, Customer_City, Grade, Opening_Amount, Recieving_Amount, Payment_Amount, and Phone. The results show three records where Payment_Amount is greater than 2000.

Customer_ID	Customer_Name	Customer_City	Grade	Opening_Amount	Recieving_Amount	Payment_Amount	Phone
4	Saqib Hassan	Lahore	65	5600	23500	54500	0344-863579
5	Tayyaba Khan	Multan	87	3400	4500	5400	0300-765367

- Less than 2000:

Limit to 1000 rows

```

44
45 • SELECT
46 *
47 FROM
48 customer
49 WHERE
50 Payment_Amount < 2000;
51

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: ☐

	Customer_ID	Customer_Name	Customer_City	Grade	Opening_Amount	Receiving_Amount	Payment_Amount	Phone
▶	1	Sumaira Safeer	Islamabad	34	4300	1550	450	0333-754468
	3	Hannah Zoe	Mianwali	23	76000	6500	900	0333-753957
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Greater than or equal to 2000:

Limit to 1000 rows

```

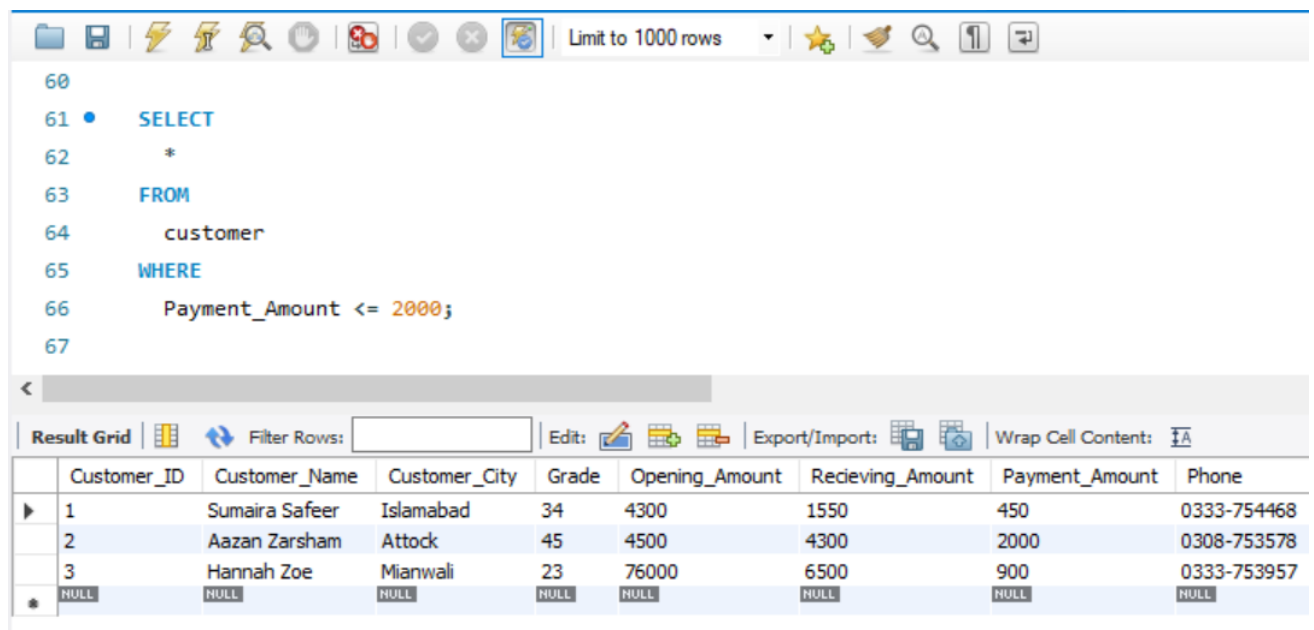
52
53 • SELECT
54 *
55 FROM
56 customer
57 WHERE
58 Payment_Amount >= 2000;
59

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: ☐

	Customer_ID	Customer_Name	Customer_City	Grade	Opening_Amount	Receiving_Amount	Payment_Amount	Phone
▶	2	Aazan Zarsham	Attock	45	4500	4300	2000	0308-753578
	4	Saqib Hassan	Lahore	65	5600	23500	54500	0344-863579
	5	Tayyaba Khan	Multan	87	3400	4500	5400	0300-765367
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Less than or equal to 2000:



The screenshot shows a database query editor with a toolbar at the top. The SQL query is as follows:

```

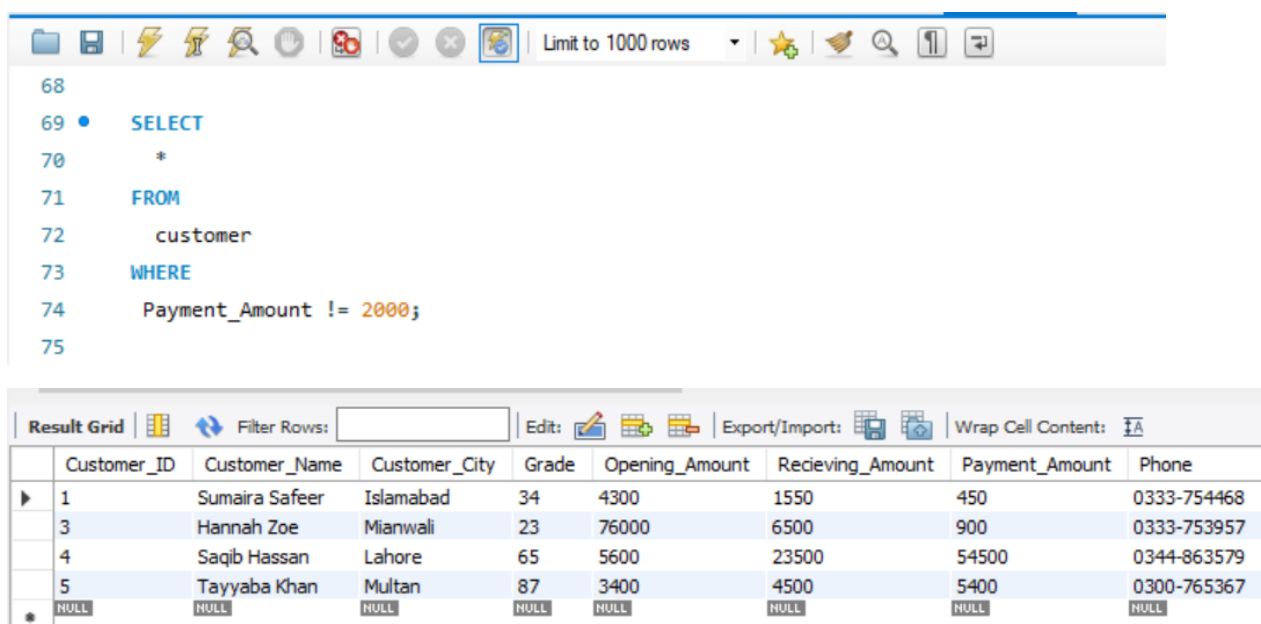
60
61 • SELECT
62     *
63 FROM
64     customer
65 WHERE
66     Payment_Amount <= 2000;
67

```

Below the query editor is the 'Result Grid' tab. It contains a table with the following data:

	Customer_ID	Customer_Name	Customer_City	Grade	Opening_Amount	Receiving_Amount	Payment_Amount	Phone
▶	1	Sumaira Safeer	Islamabad	34	4300	1550	450	0333-754468
	2	Aazan Zarsham	Attock	45	4500	4300	2000	0308-753578
	3	Hannah Zoe	Mianwali	23	76000	6500	900	0333-753957
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Not equal to 2000:



The screenshot shows a database query editor with a toolbar at the top. The SQL query is as follows:

```

68
69 • SELECT
70     *
71 FROM
72     customer
73 WHERE
74     Payment_Amount != 2000;
75

```

Below the query editor is the 'Result Grid' tab. It contains a table with the following data:

	Customer_ID	Customer_Name	Customer_City	Grade	Opening_Amount	Receiving_Amount	Payment_Amount	Phone
▶	1	Sumaira Safeer	Islamabad	34	4300	1550	450	0333-754468
	3	Hannah Zoe	Mianwali	23	76000	6500	900	0333-753957
	4	Saqib Hassan	Lahore	65	5600	23500	54500	0344-863579
	5	Tayyaba Khan	Multan	87	3400	4500	5400	0300-765367
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Task 6:

Display customer id, customer name, customer city and grade where customer city is New York and grade is 2.

```

80 • SELECT
81     Customer_ID,
82     Customer_Name,
83     Customer_City,
84     Grade
85 FROM
86     customer
87 WHERE
88     Customer_City = 'New York' AND Grade = 2;

```

Result Grid

Customer_ID	Customer_Name	Customer_City	Grade
2	Aazan Zarsham	New York	2
NULL	NULL	NULL	NULL

Task 7:

Display all records of the customer where grade is not greater than 2.

```

90
91 • SELECT
92     *
93 FROM
94     customer
95 WHERE
96     Grade <= 2;
97

```

Result Grid

Customer_ID	Customer_Name	Customer_City	Grade	Opening_Amount	Receiving_Amount	Payment_Amount	Phone
2	Aazan Zarsham	New York	2	4500	4300	2000	0308-753578
4	Saqib Hassan	New York	1	5600	23500	54500	0344-863579
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

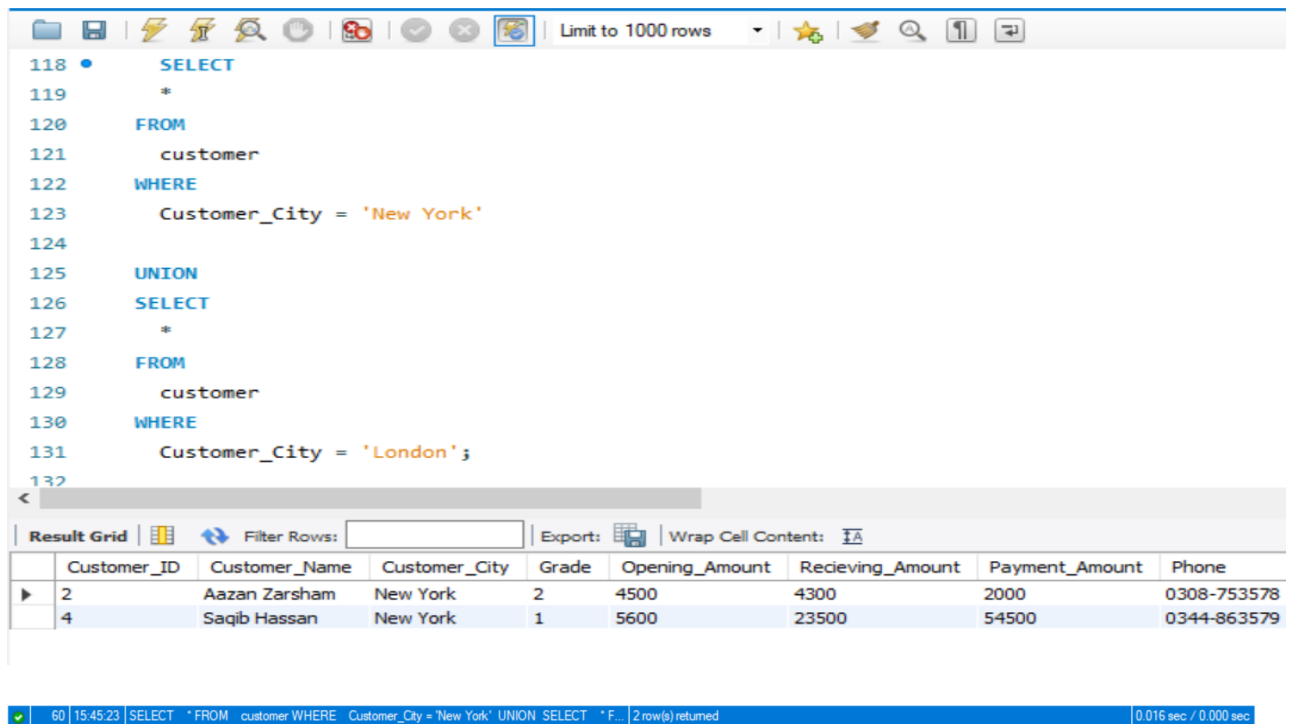
Task 8:

Display all from customer where receiving amount is in between the 5000 and 10000.

Union: Returns all distinct rows selected by both the queries.

Syntax:

SELECT column_name(s) FROM table1 UNION SELECT column_name(s) FROM table2;



The screenshot shows a SQL query editor with a query window and a result grid. The query is as follows:

```
118 SELECT
119 *
120 FROM
121 customer
122 WHERE
123 Customer_City = 'New York'
124
125 UNION
126 SELECT
127 *
128 FROM
129 customer
130 WHERE
131 Customer_City = 'London';
132
```

The result grid displays the following data:

Customer_ID	Customer_Name	Customer_City	Grade	Opening_Amount	Receiving_Amount	Payment_Amount	Phone
2	Aazan Zarsham	New York	2	4500	4300	2000	0308-753578
4	Saqib Hassan	New York	1	5600	23500	54500	0344-863579

The status bar at the bottom indicates: 60 | 15:45:23 | SELECT * FROM customer WHERE Customer_City = 'New York' UNION SELECT * F... | 2 row(s) returned | 0.016 sec / 0.000 sec

Task 11:

Display customer name from customer and customer1.

Union all:

Returns all rows selected by either query including the duplicates. Display all records from customer and customer1.

Syntax:

SELECT column_name(s) FROM table1 UNION ALL SELECT column_name(s) FROM table2;

Limit to 1000 rows

```
134 • SELECT
135     Customer_Name
136 FROM
137     customer
138
139 UNION
140
141 SELECT
142     Customer_Name
143 FROM
144     customer1;
145
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Customer_Name
▶	Sumaira Safeer
	Aazan Zarsham
	Hannah Zoe
	Saqib Hassan
	Tayyaba Khan
	Sadia Malik
	Hamza Khan
	Dua Fatima
	Kamran Hassan

62 15:56:57 SELECT Customer_Name FROM customer UNION SELECT Customer_Name FROM ... 5 row(s) returned 0.000 sec / 0.000 sec

Post lab Tasks:

Consider the following Products Table and OrderDetail.

Products Table:

	Table Name: product	Schema: customers
	Charset/Collation: utf8mb4 utf8mb4_0900_ai_c	Engine: InnoDB
	Comments:	

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	D
 Product_ID	INT	☒	☒	☐	☐	☐	☐	☐	☐	
 Product_Name	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
 Supplier_ID	INT	☐	☒	☐	☐	☐	☐	☐	☐	
 Category_ID	INT	☐	☒	☐	☐	☐	☐	☐	☐	
		☐	☐	☐	☐	☐	☐	☐	☐	

Insertion of data:

Result Grid		 Filter Rows:		Edit: 		
	Product_ID	Product_Name	Supplier_ID	Category_ID		
	10	Perfumes	11	101		
	20	Hoddies	22	102		
	30	Make Up	33	103		
	40	Shoes	44	104		
	50	Electronic Hub	55	105		
	60	Colo Crux	66	106		
	70	Venture Clothing	77	107		
	80	Sweaters	88	108		
	90	Bags	99	109		
	100	Smart Watches	100	110		
	NULL	NULL	NULL	NULL		

OrderDetail Table:


 Table Name: Schema: **customers**
 Charset/Collation: Engine:
 Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
 Oder_DetailsID	INT	☒	☒	☐	☐	☐	☐	☐	☐	
 Oder_ID	INT	☐	☒	☐	☐	☐	☐	☐	☐	
 Product_ID	INT	☐	☒	☐	☐	☐	☐	☐	☐	
 Quantity	INT	☐	☒	☐	☐	☐	☐	☐	☐	
		☐	☐	☐	☐	☐	☐	☐	☐	

Insertion of data:

Result Grid |  Filter Rows: | Edit:   

	Oder_DetailsID	Oder_ID	Product_ID	Quantity
▶	1	100	10	4
	2	200	30	8
	3	300	50	2
	4	400	20	12
	5	500	40	16
	6	600	60	287
	7	700	70	34
	8	800	80	24
	9	900	90	14
	10	1000	100	259
	11	1010	110	500
	12	1020	120	123
✱	NULL	NULL	NULL	NULL

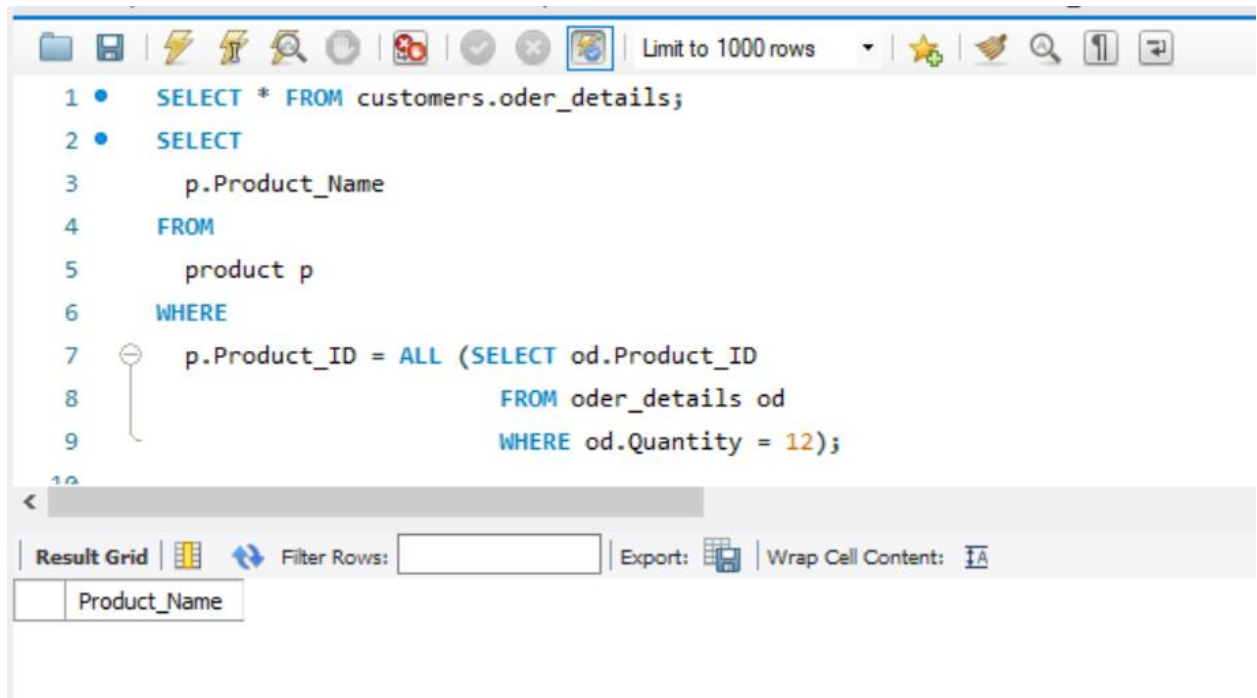
Customer Table:


 Table Name: Schema: **customers**
 Charset/Collation: Engine:
 Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	De
 Customer_Name	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
 Customer_City	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
 Customer_ID	INT	☒	☒	☐	☐	☐	☐	☐	☐	
		☐	☐	☐	☐	☐	☐	☐	☐	

Task 1:

Find the name of the product if all the records in the OrderDetails has Quantity equal to 12 (using All operator).



```
1 • SELECT * FROM customers.oder_details;
2 • SELECT
3     p.Product_Name
4 FROM
5     product p
6 WHERE
7     p.Product_ID = ALL (SELECT od.Product_ID
8                          FROM oder_details od
9                          WHERE od.Quantity = 12);
```

The screenshot shows a SQL query editor with a toolbar at the top containing icons for file operations, execution, and search. The query is written in a multi-line editor with line numbers 1 through 10. The query itself is:
1. `SELECT * FROM customers.oder_details;`
2. `SELECT`
3. `p.Product_Name`
4. `FROM`
5. `product p`
6. `WHERE`
7. `p.Product_ID = ALL (SELECT od.Product_ID`
8. `FROM oder_details od`
9. `WHERE od.Quantity = 12);`
10.
Below the query editor, there is a 'Result Grid' section with a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. A table with one column 'Product_Name' is visible below these controls.

Task 2:

Lists the product names if it finds ANY records in the OrderDetails table that quantity > 99 (using ANY operator).

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```

12
13 • SELECT
14     p.Product_Name
15 FROM
16     product p
17 WHERE
18     p.Product_ID = ANY (SELECT od.Product_ID Z
19                          FROM oder_details od
20                          WHERE od.Quantity > 99);
21

```

Below the query editor is a 'Result Grid' section. It includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The result grid shows a table with the following data:

Product_Name
Colo Crux
Smart Watches

The status bar at the bottom indicates: 105 | 20:32:22 | SELECT p.Product_Name FROM product p WHERE p.Product_ID = ANY (SELE... | 2 row(s) returned | 0.000 sec / 0.000 sec

Task 3:

Display OrderID and Quantity, If Quantity is greater than 10 then return "The amount is greater than 10" If Quantity is equal to 10 then return "The amount is 10" If Quantity is less than 10 then return "The amount is under 10" The new field name should be 'Profit'(using case statement).

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```

21 • SELECT
22     Oder_ID,
23     Quantity,
24     CASE
25         WHEN Quantity > 10 THEN 'The amount is greater than 10'
26         WHEN Quantity = 10 THEN 'The amount is 10'
27         ELSE 'The amount is under 10'
28     END AS Profit
29 FROM
30     Oder_details;
31

```

The status bar at the bottom indicates: Limit to 1000 rows

Result Grid			
		Filter Rows:	
		Export:	
		Wrap Cell Content:	
	Oder_ID	Quantity	Profit
▶	100	4	The amount is under 10
	200	8	The amount is under 10
	300	2	The amount is under 10
	400	12	The amount is greater than 10
	500	16	The amount is greater than 10
	600	287	The amount is greater than 10
	700	34	The amount is greater than 10
	800	24	The amount is greater than 10
	900	14	The amount is greater than 10
	1000	259	The amount is greater than 10
	1010	500	The amount is greater than 10
	1020	123	The amount is greater than 10

✓	104	20:30:26	SELECT Oder_ID, Quantity, CASE WHEN Quantity > 10 THEN The amount is g...	12 row(s) returned	0.000 sec / 0.000 sec
---	-----	----------	---	--------------------	-----------------------